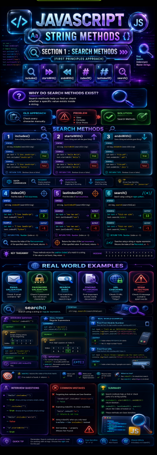
Hitesh Rathore (HR)


JAVASCRIPT STRING METHODS
SECTION 1 : SEARCH METHODS
(FIRST PRINCIPLES APPROACH)

Find. Search. Understand. Master Strings.

1 includes() 2 startsWith() 3 endsWith() 4 indexOf() 5 lastIndexOf() 6 search()

WHY DO SEARCH METHODS EXIST?

Search methods help us find or check whether a specific value exists inside a string.

OLD APPROACH Check every character manually.

PROBLEM Slow, Complex, Error Prone

SOLUTION Search Methods

SEARCH METHODS

1 includes()

2 startsWith()

3 endsWith()

4 indexOf()

5 lastIndexOf()

6 search()

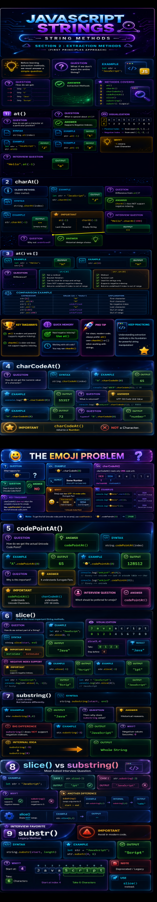
REAL WORLD EXAMPLES

INTERVIEW QUESTIONS

COMMON MISTAKES

SUMMARY

QUICK TIP


JAVASCRIPT STRINGS
STRING METHODS
SECTION 2 : EXTRACTION METHODS
(FIRST PRINCIPLES APPROACH)

Before learning extraction methods, we must answer a simple question.

QUESTION What if we don't want the entire String?

EXAMPLE
```
let str = "JavaScript";
```

ANSWER Extraction Methods

METHODS COVERED
1. at()
2. charAt()
3. charCodeAt()
4. codePointAt()
5. slice()
6. substring()
7. substr() (Legacy)

1) at()

INTERVIEW QUESTION
```
"Hello".at(-1)
```
OUTPUT "o"

2 charAt()

3 at() vs []

KEY TAKEAWAYS **QUICK MEMORY** **PRO TIP** **KEEP PRACTICING**

4 charCodeAt()

IMPORTANT charCodeAt() returns a Number. NOT a Character.

THE EMOJI PROBLEM

5 codePointAt()

6 slice()

7 substring()

8 slice() vs substring()

Most Asked Interview Question.

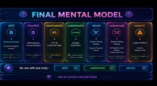
9 substr()

Legacy Method.

IMPORTANT Avoid in modern code.

NOTE Deprecated / Legacy

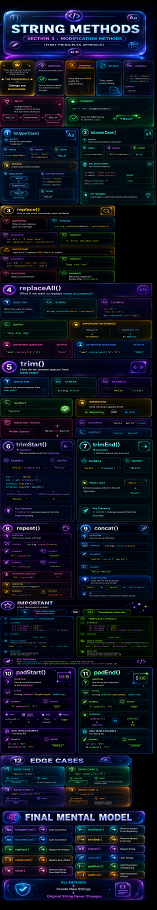
USE slice() instead.


FINAL MENTAL MODEL

at() charAt() charCodeAt() codePointAt() slice() substring() substr()

So we will use only : at() codePointAt() slice()

END OF EXTRACTION METHODS


STRING METHODS
SECTION 3 : MODIFICATION METHODS
(FIRST PRINCIPLES APPROACH)

THE GOLDEN RULE Strings are Immutable.

QUESTION What does immutable mean?

ANSWER Once a String is created, its contents cannot be changed.

IMPORTANT CONSEQUENCE String Methods NEVER modify the original String.

INSTEAD They create a NEW String.

CORRECT
```
str = str.toUpperCase();
```
Now the NEW String is stored in str. "HELLO"

1 toUpperCase()

2 toLowerCase()

3 replace() One of the most commonly used methods.

SYNTAX
```
string.replace(search, replacement)
```

EXAMPLE
```
let str = "I love Java";
str.replace("Java", "JavaScript")
```
OUTPUT "I love JavaScript"

IMPORTANT replace() replaces ONLY the first match.

4 replaceAll() What if we want to replace every occurrence?

5 trim() How do we remove spaces from both ends?

6 trimStart()

7 trimEnd()

8 repeat()

9 concat()

IMPORTANT Most developers prefer: Concatenation (Plus Operator) OR Template Literals

10 padStart()

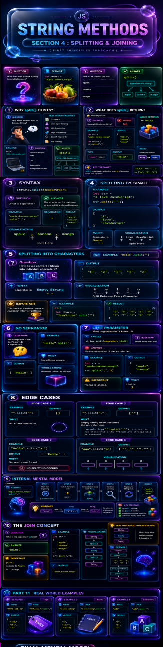
11 padEnd()

12 EDGE CASES

FINAL MENTAL MODEL

toUpperCase() → Case Conversion	trimStart() → Remove Spaces From Beginning
toLowerCase() → Case Conversion	trimEnd() → Remove Spaces From End
replace() → Replace First Match	repeat() → Repeat String
replaceAll() → Replace Every Match	concat() → Join Strings
trim() → Remove Spaces From Both Ends	padStart() → Add Characters At Beginning
	padEnd() → Add Characters At End

ALL METHODS
Create New Strings
Original String Never Changes


STRING METHODS
SECTION 4 : SPLITTING & JOINING
(FIRST PRINCIPLES APPROACH)

QUESTION What if we want to break a String into multiple pieces?

EXAMPLE
```
let fruits = "apple,banana,mango";
```

QUESTION How do we convert this into: apple banana mango

ANSWER split()

1 WHY split() EXISTS?

2 WHAT DOES split() RETURN?

3 SYNTAX
```
string.split(separator)
```

QUESTION What is separator?

ANSWER The character (or pattern) where splitting should happen.

4 SPLITTING BY SPACE

EXAMPLE
```
let str =
"I love JavaScript";
str.split(" ");
```

OUTPUT
```
[
  "I",
  "love",
  "JavaScript"
]
```

5 SPLITTING INTO CHARACTERS EXAMPLE `"Hello".split("")`

OUTPUT
```
[ "H", "e", "l", "l", "o" ]
```

6 NO SEPARATOR

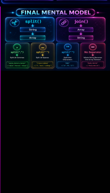
7 LIMIT PARAMETER Most beginners don't know this.

8 EDGE CASES

9 INTERNAL MENTAL MODEL

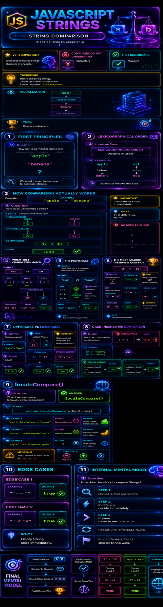
10 THE JOIN CONCEPT

PART 11 REAL WORLD EXAMPLES


FINAL MENTAL MODEL

split() String → Array

join() Array → String


JAVASCRIPT STRINGS
STRING COMPARISON
(FIRST PRINCIPLES APPROACH)

VERY IMPORTANT JavaScript compares Strings character by character.

COMPUTERS DO NOT UNDERSTAND Characters

THEY UNDERSTAND Numbers

THEREFORE Before comparing Strings, JavaScript converts comparison into a comparison of Unicode values.

THEN Comparison happens.

1 FIRST PRINCIPLES

Question: How can a computer compare: "apple" "banana" ?

We need a clear, logical way to compare strings.

2 LEXICOGRAPHICAL ORDER

Important Term: LEXICOGRAPHICAL ORDER Dictionary Order

JavaScript follows this idea.

3 HOW COMPARISON ACTUALLY WORKS

EXAMPLE `"apple" < "banana"`

QUESTION How does JavaScript decide?

STEP 1 Compare first characters.

COMPARISON `97 < 98`

RESULT `true`

IMPORTANT Comparison stops immediately.

WHY? First difference found.

4 WHEN FIRST CHARACTERS MATCH

5 THE PREFIX RULE

6 THE MOST FAMOUS INTERVIEW QUESTION `"2" > "15"`

7 UPPERCASE VS LOWERCASE

8 CASE-INSENSITIVE COMPARISON

9 localeCompare()

Question: What if we want proper language-aware comparison?

ANSWER `localeCompare()`

SYNTAX
```
string.localeCompare(otherString)
```

10 EDGE CASES

EDGE CASE 1: EXAMPLE `"" === ""` OUTPUT `true`

EDGE CASE 2: EXAMPLE `"" < "a"` OUTPUT `true`

WHY? Empty String ends immediately.

11 INTERNAL MENTAL MODEL

Question: How does JavaScript compare Strings?

STEP 1 Compare first characters.

STEP 2 If different, decide immediately.

STEP 3 If same, move to next character.

Repeat until difference found.

If no difference found, shorter String wins.

FINAL MENTAL MODEL